TeeTime Golf App

Backend Architecture Documentation

Development Team

November 23, 2025

Contents

1 Executive Summary

The TeeTime Golf App backend is a production-grade Node.js/Express API server designed to handle golf game coordination, user authentication, and social features. This document provides comprehensive coverage of the implemented architecture, security measures, and technical decisions.

1.1 Key Achievements

- Complete JWT-based authentication system with refresh tokens
- Phone number and email-based user registration
- Magic link authentication for passwordless login
- Device fingerprinting and session management
- Comprehensive analytics and audit logging
- Production-ready security measures
- Scalable PostgreSQL database architecture

2 System Architecture

2.1 Technology Stack

- **Runtime**: Node.js 18+
- **Framework**: Express.js 4.x
- **Database**: PostgreSQL (Supabase hosted)
- **Authentication**: JWT with bcrypt password hashing
- **Email Service**: SendGrid API
- **SMS Service**: Twilio API
- **Logging**: Winston with structured logging
- **Security**: Helmet, CORS, Rate limiting

2.2 Project Structure

```
backend/
          config/
                database.js      # PostgreSQL connection
                logger.js        # Winston logging setup
          controllers/
                authController.js # Authentication logic
          middleware/
```

```
                auth.js           # JWT verification
                analytics.js      # Event logging
                errorHandler.js   # Error management
                rateLimiter.js    # Rate limiting
                validator.js      # Input validation
        repositories/
                userRepository.js
                sessionRepository.js
                otpRepository.js
                magicLinkRepository.js
                deviceRepository.js
        routes/
                auth.js           # Authentication routes
        utils/
                crypto.js         # JWT and hashing utilities
                validation.js     # Input validation helpers
                deviceFingerprint.js # Device tracking
        server.js             # Main application entry
```

3 Database Schema

3.1 Core Tables

3.1.1 Users Table

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    phone_number VARCHAR(20) UNIQUE,
    email VARCHAR(255) UNIQUE,
    username VARCHAR(50) UNIQUE,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    password_hash VARCHAR(255),
    handicap INTEGER,
    social_rate INTEGER DEFAULT 100,
    avatar_url TEXT,
    is_active BOOLEAN DEFAULT true,
    is_verified BOOLEAN DEFAULT false,
    is_locked BOOLEAN DEFAULT false,
    failed_login_attempts INT DEFAULT 0,
    locked_until TIMESTAMPTZ,
    timezone VARCHAR(50) DEFAULT 'UTC',
    language VARCHAR(10) DEFAULT 'en',
    notification_preferences JSONB DEFAULT
        '{"email": true, "sms": true, "push": true}'::jsonb,
    last_login_at TIMESTAMPTZ,
    last_login_ip INET,
    last_login_device_id INTEGER,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
```

```
    deleted_at TIMESTAMPTZ
);
```

3.1.2 Sessions Table

```
CREATE TABLE sessions (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE
       CASCADE,
    device_id INTEGER REFERENCES devices(id) ON DELETE CASCADE,
    access_token_hash VARCHAR(255) NOT NULL,
    refresh_token_hash VARCHAR(255) NOT NULL,
    token_family_id VARCHAR(255) NOT NULL,
    ip_address INET,
    user_agent TEXT,
    is_active BOOLEAN DEFAULT true,
    remember_me BOOLEAN DEFAULT false,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    expires_at TIMESTAMPTZ NOT NULL,
    last_activity_at TIMESTAMPTZ DEFAULT NOW(),
    revoked_at TIMESTAMPTZ,
    revoked_by INTEGER REFERENCES users(id),
    revoke_reason VARCHAR(255)
);
```

3.1.3 Additional Tables

- **devices**: Device fingerprinting and tracking
- **otp_requests**: SMS/Email OTP verification codes
- **magic_links**: Passwordless authentication tokens
- **analytics_events**: Comprehensive event logging
- **login_logs**: Authentication audit trail

4 Authentication System

4.1 Multi-Factor Authentication Options

4.1.1 Phone Number + OTP

1. User enters phone number
2. System generates 6-digit OTP
3. OTP sent via Twilio SMS (fallback to console in development)
4. User verifies OTP within 60 seconds
5. Account created with profile completion

4.1.2 Magic Link Authentication

1. User enters email address
2. System generates cryptographically secure token
3. Email sent via SendGrid with universal link
4. Link opens mobile app via deep linking
5. Automatic authentication and profile setup

4.1.3 Username + Password

1. User enters username and password
2. System verifies credentials against bcrypt hash
3. JWT tokens generated for session management
4. Device fingerprinting for security tracking

4.2 JWT Token Management

4.2.1 Access Tokens

- **Expiry**: 15 minutes (configurable)
- **Purpose**: API authentication
- **Payload**: User ID, session ID, issued time
- **Algorithm**: HS256 with secret key

4.2.2 Refresh Tokens

- **Expiry**: 7 days (configurable)
- **Purpose**: Access token renewal
- **Payload**: User ID, token family ID, session ID
- **Security**: Token rotation on each refresh

4.3 Security Measures

4.3.1 Password Security

- bcrypt hashing with salt rounds (cost factor 10)
- Minimum 6 characters (configurable)
- Failed attempt tracking with account lockout
- Password reset via OTP or magic link

4.3.2 Token Security

- Tokens stored as SHA-256 hashes in database
- Token family tracking prevents replay attacks
- Automatic token rotation on refresh
- Session revocation capabilities

4.3.3 Rate Limiting

- OTP requests: 3 per hour per phone number
- Login attempts: 5 per 15 minutes per IP
- Magic links: 3 per hour per email
- Token refresh: 10 per minute per user

5 API Endpoints

5.1 Authentication Endpoints

Endpoint	Method	Description
/auth/request-otp	POST	Request OTP for phone verification
/auth/verify-otp	POST	Verify OTP and create account
/auth/set-password	POST	Set password after OTP verification
/auth/login	POST	Login with username/phone + password
/auth/refresh	POST	Refresh access token using refresh token
/auth/logout	POST	Logout current session
/auth/logout-all	POST	Logout from all devices
/auth/me	GET	Get current user profile
/auth/magic-link	POST	Request magic link for email
/auth/verify-magic	GET	Verify magic link token
/auth/magic-link-redirect	GET	Universal link handler for mobile apps

5.2 Request/Response Examples

5.2.1 Login Request

```
POST /auth/login
{
  "username": "johndoe",
  "password": "SecurePass123!",
  "rememberMe": true
}
```

5.2.2 Login Response

```
{
  "success": true,
  "message": "Login successful.",
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": 123,
    "username": "johndoe",
    "firstName": "John",
    "lastName": "Doe",
    "email": "john@example.com",
    "phoneNumber": "+1234567890",
    "handicap": 15
  }
}
```

6 Device Fingerprinting

6.1 Captured Information

- Operating System (iOS, Android, Windows, macOS, Linux)
- Browser (Chrome, Safari, Firefox, Edge)
- Device Type (Mobile, Tablet, Desktop)
- IP Address and User Agent
- Screen Resolution and Language Preferences

6.2 Security Benefits

- Detect suspicious login attempts from new devices
- Track user activity across sessions
- Enable device-specific security policies
- Support "trusted device" functionality

7 Analytics and Logging

7.1 Event Tracking

The system automatically logs all authentication events:

- User registration and verification
- Login attempts (successful and failed)
- Token refresh operations
- Session management activities
- Magic link requests and verifications
- Device registration and updates

7.2 Structured Logging

Using Winston logger with multiple transports:

- Console output for development
- File rotation for production
- Error-level alerts for critical issues
- JSON format for log aggregation

8 Error Handling

8.1 Error Classification

- **Validation Errors**: Input format and constraint violations
- **Authentication Errors**: Invalid credentials or expired tokens
- **Authorization Errors**: Insufficient permissions
- **Rate Limit Errors**: Too many requests
- **System Errors**: Database or service failures

8.2 Error Response Format

```
{
  "success": false,
  "error": {
    "message": "Invalid credentials",
    "code": "AUTH_FAILED",
    "details": {
      "field": "username",
      "reason": "User not found"
    }
  }
}
```

9 Mobile App Integration

9.1 Deep Linking

The backend supports universal links for mobile app integration:

- Magic links automatically open the mobile app
- Fallback to web interface if app not installed
- Token verification through app-specific URLs
- Seamless user experience across platforms

9.2 React Native Compatibility

- AsyncStorage integration for token persistence
- Automatic token refresh handling
- Device-specific session management
- Push notification preparation (Firebase integration pending)

10 Firebase Implementation Status

10.1 Current Status: Not Implemented

Firebase integration for push notifications has been **intentionally deferred** during the initial development phase. This decision was made for the following strategic reasons:

10.1.1 Priority Focus

- **Core Authentication**: Establishing a robust, secure authentication system took precedence
- **Database Architecture**: Ensuring scalable data models and relationships
- **API Stability**: Creating reliable endpoints for mobile app integration
- **Security Implementation**: Implementing comprehensive security measures

10.1.2 Technical Rationale

- **Complexity Management**: Firebase integration adds significant complexity to the authentication flow
- **Testing Requirements**: Push notifications require extensive device testing across platforms
- **User Experience**: Core app functionality needed to be solid before adding notification features
- **Resource Allocation**: Limited development time required focus on essential features

10.1.3 Planned Implementation

Firebase Cloud Messaging (FCM) integration is planned for Phase 2 development:

1. **Device Token Management**: Store FCM tokens in device fingerprinting system
2. **Notification Categories**:
    - Tee time invitations
    - Game roster updates
    - Score submissions
    - Social interactions
3. **Notification Preferences**: User-configurable notification settings
4. **Background Processing**: Queue-based notification delivery

10.1.4 Current Preparation

The system is already prepared for Firebase integration:

- Device tracking infrastructure in place
- User preference system supports notification settings
- Analytics system can track notification engagement
- Database schema includes notification-related fields

11 Production Deployment

11.1 Environment Configuration

- **Development**: Local PostgreSQL, console-based OTP/email
- **Staging**: Supabase PostgreSQL, real SMS/email services
- **Production**: Full service integration with monitoring

11.2 Security Checklist

JWT secrets rotated for production

Database credentials secured

CORS origins restricted

Rate limiting configured

HTTPS enforced

Error messages sanitized

Logging configured for monitoring

11.3 Monitoring and Maintenance

- Health check endpoint for uptime monitoring
- Structured logging for error tracking
- Database connection pooling for performance
- Graceful shutdown handling
- Automatic session cleanup

12 Performance Considerations

12.1 Database Optimization

- Indexed columns for fast lookups (phone, email, username)
- Connection pooling for concurrent requests
- Query optimization for session validation
- Automatic cleanup of expired tokens

12.2 Caching Strategy

- JWT verification caching (future implementation)
- User profile caching for frequent access
- Rate limiting counters in memory
- Session validation optimization

13 Testing Strategy

13.1 Implemented Testing

- Manual API testing with curl commands
- Database schema validation
- Authentication flow testing
- Error handling verification

13.2 Recommended Additional Testing

- Unit tests for utility functions
- Integration tests for API endpoints
- Load testing for concurrent users
- Security penetration testing

14 Future Enhancements

14.1 Phase 2 Features

- Firebase Cloud Messaging integration
- Social features (friend requests, groups)
- Golf course integration
- Tee time booking system
- Score tracking and handicap calculation

14.2 Technical Improvements

- Database migration system (Knex/Prisma)
- API documentation (Swagger/OpenAPI)
- Automated testing suite
- CI/CD pipeline
- Performance monitoring

15 Conclusion

The TeeTime Golf App backend represents a comprehensive, production-ready authentication and user management system. The architecture prioritizes security, scalability, and maintainability while providing a solid foundation for future feature development.

Key achievements include:

- Robust multi-factor authentication system
- Comprehensive security measures
- Scalable database architecture
- Mobile app integration capabilities
- Production-ready deployment structure

The strategic decision to defer Firebase integration allows for a focused, stable core system that can be enhanced with push notifications in subsequent development phases.

A Configuration Files

A.1 Environment Variables

```
# Database
DATABASE_URL=postgresql://user:pass@host:port/db

# JWT Configuration
JWT_SECRET=your-secret-key-here
JWT_REFRESH_SECRET=your-refresh-secret-here
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d

# External Services
TWILIO_ACCOUNT_SID=ACxxxxx
TWILIO_AUTH_TOKEN=xxxxx
SENDGRID_API_KEY=SG.xxxxx
SENDGRID_FROM_EMAIL=noreply@teetimeapp.com

```

```
# Application
PORT=3000
NODE_ENV=production
API_URL=https://api.teetimeapp.com
```

B Database Migration Scripts

B.1 Initial Schema

Refer to `backend/db/schema.sql` for complete database schema.

B.2 Migration Commands

```
# Setup database
node setup-database.js

# Run migrations
node migrate-database.js

# Verify schema
node -e "require('./config/database').testConnection()"
```